Data Mining

SUPPORT VECTOR MACHINES

BY: IVETA MRÁZOVÁ

Introduction to SVM

Support vector machines were invented by V. Vapnik and his co-workers in 1970s in Russia and became known to the West in 1992.

SVMs are linear classifiers that find a hyperplane to separate two class of data, positive and negative.

Kernel functions are used for nonlinear separation.

SVM not only has a rigorous theoretical foundation, but also performs classification more accurately than most other methods in applications, especially for high dimensional data.

It belongs to the best classifiers for text classification.

Basic concepts

Let the set of training examples D be

$$\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_r, y_r)\},$$

where $\mathbf{x}_i = (x_1, x_2, \ldots, x_n)$ is an **input vector** in a real-valued space $X \subseteq R^n$ and y_i is its **class label** (output value), $y_i \in \{1, -1\}$.

1: positive class and -1: negative class.

SVM finds a linear function of the form (**w**: weight vector)

$$f(\mathbf{x}) = \langle \mathbf{w} \cdot \mathbf{x} \rangle + b$$

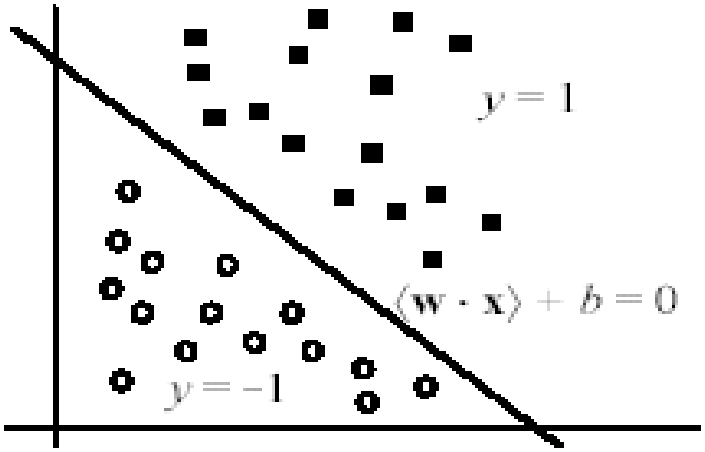
$$y_i = \begin{cases} 1 & if \ \langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b \geq 0 \\ -1 & if \ \langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b < 0 \end{cases}$$

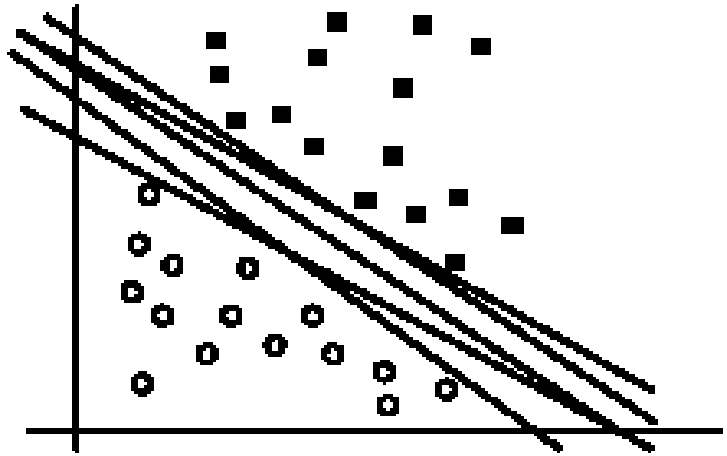
The hyperplane

The hyperplane that separates positive and negative training data is $\langle \mathbf{w} \cdot \mathbf{x} \rangle + b = 0$

It is also called the decision boundary (surface).

So many possible hyperplanes, which one to choose?

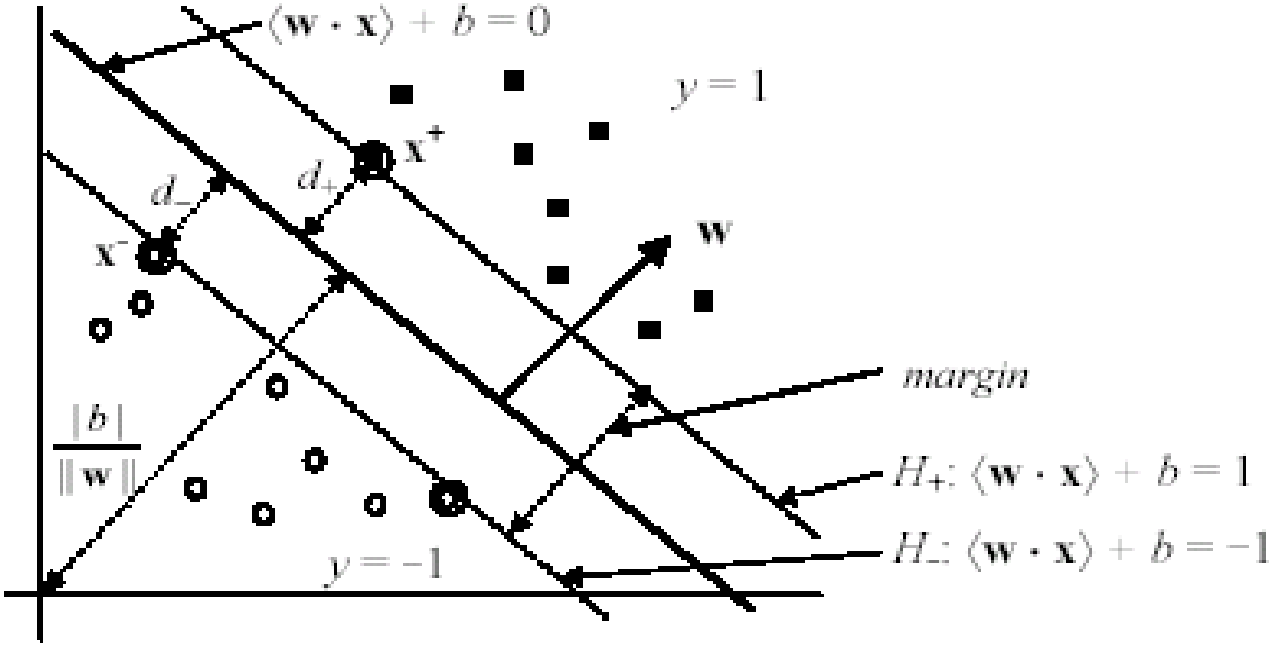




Maximal margin hyperplane

SVM looks for the separating hyperplane with the largest margin.

Machine learning theory says this hyperplane minimizes the error bound



Linear SVM: separable case

Assume the data are linearly separable.

Consider a positive data point ($\mathbf{x}^+$, 1) and a negative ($\mathbf{x}^-$, -1) that are closest to the hyperplane

$$\langle \mathbf{w} \cdot \mathbf{x} \rangle + b = 0.$$

We define two parallel hyperplanes, H_+ and H_-, that pass through $\mathbf{x}^+$ and $\mathbf{x}^-$ respectively. H_+ and H_- are also parallel to $\langle \mathbf{w} \cdot \mathbf{x} \rangle + b = 0$.

$$H_+: \quad \langle \mathbf{w} \cdot \mathbf{x}^+ \rangle + b = 1$$

$$H_-: \quad \langle \mathbf{w} \cdot \mathbf{x}^- \rangle + b = -1$$

$$\text{such that} \quad \begin{cases} \langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b \geq 1 & \text{if } y_i = 1 \\ \langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b \leq -1 & \text{if } y_i = -1, \end{cases}$$

Compute the margin

Now let us compute the distance between the two margin hyperplanes H_+ and H_-. Their distance is the **margin** ($d_+ + d_-$ in the figure).

Recall from vector space in algebra that the (perpendicular) **distance** from a point $\mathbf{x}_i$ to the hyperplane $\langle \mathbf{w} \cdot \mathbf{x} \rangle + b = 0$ is:

$$\frac{|\langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b|}{\|\mathbf{w}\|} \tag{36}$$

where $||\mathbf{w}||$ is the norm of $\mathbf{w}$,

$$\|\mathbf{w}\| = \sqrt{<\mathbf{w} \cdot \mathbf{w}>} = \sqrt{w_1^2 + w_2^2 + \ldots + w_n^2} \tag{37}$$

Compute the margin (cont ...)

Let us compute d_+.

Instead of computing the distance from $\mathbf{x}^+$ to the separating hyperplane $\langle \mathbf{w} \cdot \mathbf{x} \rangle + b = 0$, we pick up any point $\mathbf{x}_s$ on $\langle \mathbf{w} \cdot \mathbf{x} \rangle + b = 0$ and compute the distance from $\mathbf{x}_s$ to $\langle \mathbf{w} \cdot \mathbf{x}^+ \rangle + b = 1$ by applying the distance Eq. (36) and noticing $\langle \mathbf{w} \cdot \mathbf{x}_s \rangle + b = 0$,

$$d_+ = \frac{|\langle \mathbf{w} \cdot \mathbf{x_s} \rangle + b - 1|}{\| \mathbf{w} \|} = \frac{1}{\| \mathbf{w} \|} \tag{38}$$

$$margin = d_+ + d_- = \frac{2}{\| \mathbf{w} \|} \tag{39}$$

A optimization problem!

Definition (Linear SVM: separable case): Given a set of linearly separable training examples,

$$D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_r, y_r)\}$$

Learning is to solve the following constrained minimization problem,

$$\text{Minimize}: \quad \frac{\langle \mathbf{w} \cdot \mathbf{w} \rangle}{2}$$

$$\text{Subject to}: \quad y_i(\langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b) \geq 1, \quad i = 1, 2, \ldots, r \qquad (40)$$

$y_i(\langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b \geq 1, \quad i = 1, 2, \ldots, r$ summarizes

$\langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b \geq 1$ for $y_i = 1$

$\langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b \leq -1$ for $y_i = -1$.

Solve the constrained minimization

Standard Lagrangian method

$$L_P = \frac{1}{2}\langle \mathbf{w} \cdot \mathbf{w} \rangle - \sum_{i=1}^{r} \alpha_i [y_i (\langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b) - 1] \qquad (41)$$

where $\alpha_i \geq 0$ are the **Lagrange multipliers**.

Optimization theory says that an optimal solution to (41) must satisfy certain conditions, called **Kuhn-Tucker conditions**, which are necessary (but not sufficient)

Kuhn-Tucker conditions play a central role in constrained optimization.

Kuhn-Tucker conditions

$$\frac{\partial L_P}{\partial w_j} = w_j - \sum_{i=1}^{r} y_i \alpha_i \mathbf{x}_i = 0, \quad j = 1, 2, \ldots, m \tag{48}$$

$$\frac{\partial L_P}{\partial b} = -\sum_{i=1}^{r} y_i \alpha_i = 0 \tag{49}$$

$$y_i(\langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b) - 1 \geq 0, \quad i = 1, 2, \ldots, r \tag{50}$$

$$\alpha_i \geq 0, \quad i = 1, 2, \ldots, r \tag{51}$$

$$\alpha_i (y_i(\langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b) - 1) = 0, \quad i = 1, 2, \ldots, r \tag{52}$$

Eq. (50) is the original set of constraints.

The complementarity condition (52) shows that only those data points on the margin hyperplanes (i.e., H_+ and H_-) can have $\alpha_i > 0$ since for them $y_i(\langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b) - 1 = 0$.

These points are called the **support vectors**, All the other parameters $\alpha_i = 0$.

Solve the problem

In general, Kuhn-Tucker conditions are necessary for an optimal solution, but not sufficient.

However, for our minimization problem with a convex objective function and linear constraints, the Kuhn-Tucker conditions are both **necessary** and **sufficient** for an optimal solution.

Solving the optimization problem is still a difficult task due to the inequality constraints.

However, the Lagrangian treatment of the convex optimization problem leads to an alternative **dual** formulation of the problem, which is easier to solve than the original problem (called the **primal**).

Dual formulation

From primal to a dual: Setting to zero the partial derivatives of the Lagrangian (41) with respect to the **primal variables** (i.e., **w** and *b*), and substituting the resulting relations back into the Lagrangian.

- I.e., substitute (48) and (49), into the original Lagrangian (41) to eliminate the primal variables

$$L_D = \sum_{i=1}^{r} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{r} y_i y_j \alpha_i \alpha_j \langle \mathbf{x}_i \cdot \mathbf{x}_j \rangle, \tag{55}$$

Dual optimization problem

$$\text{Maximize: } L_D = \sum_{i=1}^{r} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{r} y_i y_j \alpha_i \alpha_j \langle \mathbf{x}_i \cdot \mathbf{x}_j \rangle. \tag{56}$$

$$\text{Subject to: } \sum_{i=1}^{r} y_i \alpha_i = 0$$

$$\alpha_i \geq 0, \quad i = 1, 2, \ldots, r.$$

- This dual formulation is called the **Wolfe dual**.
- For the convex objective function and linear constraints of the primal, it has the property that the maximum of L_D occurs at the same values of **w**, b and α_i, as the minimum of L_P (the primal).
- Solving (56) requires numerical techniques and clever strategies, which are beyond our scope.

The final decision boundary

After solving (56), we obtain the values for α_i, which are used to compute the weight vector **w** and the bias *b* using Equations (48) and (52) respectively.

The decision boundary

$$\langle \mathbf{w} \cdot \mathbf{x} \rangle + b = \sum_{i \in sv} y_i \alpha_i \langle \mathbf{x}_i \cdot \mathbf{x} \rangle + b = 0 \qquad (57)$$

Testing: Use (57). Given a test instance **z**,

$$sign(\langle \mathbf{w} \cdot \mathbf{z} \rangle + b) = sign\left(\sum_{i \in sv} \alpha_i y_i \langle \mathbf{x}_i \cdot \mathbf{z} \rangle + b \right) \qquad (58)$$

If (58) returns 1, then the test instance **z** is classified as positive; otherwise, it is classified as negative.

How to deal with nonlinear separation?

The SVM formulations require linear separation.

Real-life data sets may need nonlinear separation.

To deal with nonlinear separation, the same formulation and techniques as for the linear case are still used.

We only transform the input data into another space (usually of a much higher dimension) so that

- a linear decision boundary can separate positive and negative examples in the transformed space,

The transformed space is called the **feature space**. The original data space is called the **input space**.

Space transformation

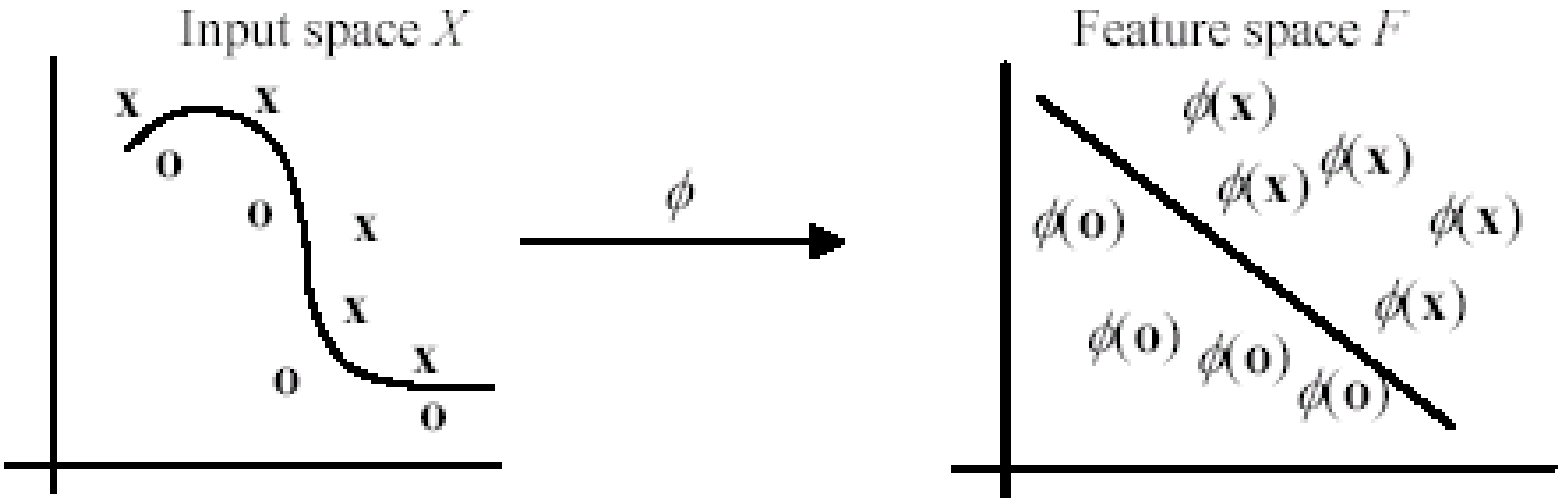
The basic idea is to map the data in the input space X to a feature space F via a nonlinear mapping ϕ,

$$\begin{aligned} \phi &: X \to F \\ \mathbf{x} &\mapsto \phi(\mathbf{x}) \end{aligned} \tag{76}$$

After the mapping, the original training data set $\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_r, y_r)\}$ becomes:

$$\{(\phi(\mathbf{x}_1), y_1), (\phi(\mathbf{x}_2), y_2), \ldots, (\phi(\mathbf{x}_r), y_r)\} \tag{77}$$

Geometric interpretation



- In this example, the transformed space is also 2-D. But usually, the number of dimensions in the feature space is much higher than that in the input space

Optimization problem in (40) becomes

With the transformation, the optimization problem in (40) becomes

$$\text{Minimize}: \quad \frac{\langle \mathbf{w} \cdot \mathbf{w} \rangle}{2} \tag{78}$$

$$\text{Subject to}: \quad y_i(\langle \mathbf{w} \cdot \phi(\mathbf{x}_i) \rangle + b) \geq 1 \qquad i = 1, 2, \ldots, r$$

The dual is

$$\text{Maximize}: \quad L_D = \sum_{i=1}^{r} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{r} y_i y_j \alpha_i \alpha_j \langle \phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}_j) \rangle. \tag{79}$$

$$\text{Subject to}: \quad \sum_{i=1}^{r} y_i \alpha_i = 0$$

$$0 \leq \alpha_i \qquad i = 1, 2, \ldots, r.$$

The final decision rule for classification (testing) is

$$\sum_{i=1}^{r} y_i \alpha_i \langle \phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}) \rangle + b \tag{80}$$

An example space transformation

Suppose our input space is 2-dimensional, and we choose the following transformation (mapping) from 2-D to 3-D:

$$(x_1, x_2) \mapsto ({x_1}^2, {x_2}^2, \sqrt{2}x_1x_2)$$

The training example ((2, 3), -1) in the input space is transformed to the following in the feature space:

((4, 9, 8.5), -1)

Problem with explicit transformation

The potential problem with this explicit data transformation and then applying the linear SVM is that it may suffer from the curse of dimensionality.

The number of dimensions in the feature space can be huge with some useful transformations even with reasonable numbers of attributes in the input space.

This makes it computationally infeasible to handle.

Fortunately, explicit transformation is not needed.

Kernel functions

We notice that in the dual formulation both

- the construction of the optimal hyperplane (79) in *F* and
- the evaluation of the corresponding decision function (80)

only require dot products $\langle \phi(\mathbf{x}) \cdot \phi(\mathbf{z}) \rangle$ and never the mapped vector $\phi(\mathbf{x})$ in its explicit form. **This is a crucial point.**

Thus, if we have a way to compute the dot product $\langle \phi(\mathbf{x}) \cdot \phi(\mathbf{z}) \rangle$ using the input vectors **x** and **z** directly,

- no need to know the feature vector $\phi(\mathbf{x})$ or even ϕ itself.

In SVM, this is done through the use of **kernel functions**, denoted by *K* :

$$K(\mathbf{x}, \mathbf{z}) = \langle \phi(\mathbf{x}) \cdot \phi(\mathbf{z}) \rangle \quad (82)$$

An example kernel function

Polynomial kernel: $K(\mathbf{x}, \mathbf{z}) = \langle \mathbf{x} \cdot \mathbf{z} \rangle^d$ (83)

Let us compute the kernel with degree $d = 2$ in a 2-dimensional space: $\mathbf{x} = (x_1, x_2)$ and $\mathbf{z} = (z_1, z_2)$.

$$\begin{aligned} \langle \mathbf{x} \cdot \mathbf{z} \rangle^2 &= (x_1 z_1 + x_2 z_2)^2 \\ &= x_1^2 z_1^2 + 2 x_1 z_1 x_2 z_2 + x_2^2 z_2^2 \\ &= \langle (x_1^2, x_2^2, \sqrt{2} x_1 x_2) \cdot (z_1^2, z_2^2, \sqrt{2} z_1 z_2) \rangle \\ &= \langle \phi(\mathbf{x}) \cdot \phi(\mathbf{z}) \rangle, \end{aligned} \quad (84)$$

This shows that the kernel $\langle \mathbf{x} \cdot \mathbf{z} \rangle^2$ is a dot product in a transformed feature space

Kernel trick

The derivation in (84) is only for illustration purposes.

We do not need to find the mapping function.

We can apply the kernel function directly by

- replace all the dot products $\langle \phi(\mathbf{x}) \cdot \phi(\mathbf{z}) \rangle$ in (79) and (80) with the kernel function $K(\mathbf{x}, \mathbf{z})$ (e.g., the polynomial kernel $\langle \mathbf{x} \cdot \mathbf{z} \rangle^d$ in (83)).

This strategy is called the **kernel trick**.

Is it a kernel function?

The question is: how do we know whether a function is a kernel without performing the derivation such as that in (84)? I.e,

- How do we know that a kernel function is indeed a dot product in some feature space?

This question is answered by a theorem called the Mercer's theorem, which we will not discuss here.

Commonly used kernels

It is clear that the idea of kernel generalizes the dot product in the input space. This dot product is also a kernel with the feature map being the identity

$$K(\mathbf{x}, \mathbf{z}) = \langle \mathbf{x} \cdot \mathbf{z} \rangle. \tag{85}$$

Commonly used kernels include

$$\text{Polynomial:} \quad K(\mathbf{x}, \mathbf{z}) = (\langle \mathbf{x} \cdot \mathbf{z} \rangle + \theta)^d \tag{86}$$

$$\text{Gaussian RBF:} \quad K(\mathbf{x}, \mathbf{z}) = e^{-\|\mathbf{x}-\mathbf{z}\|^2 / 2\sigma} \tag{87}$$

$$\text{Sigmoidal:} \quad K(\mathbf{x}, \mathbf{z}) = \tanh(k\langle \mathbf{x} \cdot \mathbf{z} \rangle - \delta) \tag{88}$$

where $\theta \in R$, $d \in N$, $\sigma > 0$, and $k, \delta \in R$.

Some other issues in SVM

SVM works only in a real-valued space. For a categorical attribute, we need to convert its categorical values to numeric values.

SVM does only two-class classification. For multi-class problems, some strategies can be applied, e.g., one-against-rest, and error-correcting output coding.

The hyperplane produced by SVM is hard to understand by human users. The matter is made worse by kernels. Thus, SVM is commonly used in applications that do not required human understanding.

Data Mining

EVALUATION OF CLASSIFIERS

BY: IVETA MRÁZOVÁ

Evaluating classification methods

- **Predictive accuracy**

$$Accuracy = \frac{Number\ of\ correct\ classifications}{Total\ number\ of\ test\ cases}$$

- Efficiency: time to construct the model
  time to use the model
- Robustness: handling noise and missing values
- Scalability: efficiency in disk-resident databases
- Interpretability: clarity and insight provided by the model
- Compactness of the model:
  - size of the tree, the number of rules, ...

Evaluation methods

- **Holdout set**: The available data set *D* is divided into two disjoint subsets,
  - the *training set* D_{train} (for learning a model)
  - the *test set* D_{test} (for testing the model)
- **Important:** training set should not be used in testing and the test set should not be used in learning.
  - Unseen test set provides an unbiased estimate of accuracy.
- The test set is also called the **holdout set**. (the examples in the original data set *D* are all labeled with classes.)
- This method is mainly used for a large data set *D*.

Evaluation methods (cont.)

***k*-fold cross-validation:**

- The available data is partitioned uniformly into ***k*** equal-size disjoint subsets.
- Use each subset as a test set and combine the remaining ***k-1*** subsets as the training set to learn a classifier.
- The procedure is run *k* times yielding ***k*** accuracies ***Accuracy*$_i$ *; 1≤i≤k***
- The final estimated accuracy $\overline{\boldsymbol{Accuracy}}$ of learning is the average of the ***k*** accuracies:

$$\overline{\boldsymbol{Accuracy}} = \frac{\mathbf{1}}{\boldsymbol{k}} \sum_{\boldsymbol{i=1}}^{\boldsymbol{k}} \boldsymbol{Accuracy_i}$$

Evaluation methods (cont.)

***k*-fold cross-validation**:

- Estimation of the approximate ***N%* confidence intervals**:

$$\overline{Accuracy} \mp t_{N,k-1} s_{\overline{Accuracy}},$$

where $s_{\overline{Accuracy}}$ is an estimate of standard deviation of the distribution governing $\overline{Accuracy}$:

$$s_{\overline{Accuracy}} = \sqrt{\frac{1}{k(k-1)} \sum_{i=1}^{k} (Accuracy_i - \overline{Accuracy})^2}$$

- Values of $t_{N,v}$ for two-sided confidence intervals:
  - often, $t_{N,v}$ is set to ***1.96*** (for ***v* = ∞** and **N = 95%**)
- ***10***-fold and ***5***-fold cross-validations are common.
- This method shall be used rather for small data.

	Confidence level *N%*		
	90%	95%	99%
v = 2	2.92	4.30	9.92
v = 10	1.81	2.23	3.17
v = 30	1.70	2.04	2.75
v = ∞	1.64	1.96	2.58

Evaluation methods (cont.)

- **Leave-one-out cross-validation**: This method is used when the data set is very small.
- It is a special case of cross-validation
- Each fold of the cross validation has only a single test example and all the rest of the data is used in training.
- If the original data has *m* examples, this is *m*-fold cross-validation

Evaluation methods (cont.)

- **Validation set**: available data is divided into 3 subsets,
  - a training set,
  - a validation set and
  - a test set.
- A validation set is used frequently for estimating parameters in learning algorithms.
- In such cases, the values that give the best accuracy on the validation set are used as the final parameter values.
- Cross-validation can be used for parameter estimating as well.

Classification measures

- Accuracy is only one measure (error = 1-accuracy).
- **Accuracy is not suitable in some applications**.
- In text mining, we may only be interested in the documents of a particular topic, which are only a small portion of a big document collection.
- In classification involving skewed or highly imbalanced data, e.g., network intrusion and financial fraud detections, we are interested only in the minority class.
  - High accuracy does not mean any intrusion is detected.
  - E.g., 1% intrusion. Achieve 99% accuracy by doing nothing.
- The class of interest is commonly called the **positive class**, and the rest **negative classes.**

Precision and recall measures

- Used in information retrieval and text classification.
- We use a confusion matrix to introduce them.

	Classified Positive	Classified Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

- ***TP*** – the number of correct classifications of the positive examples (**true positive**)
- ***FN*** – the number of incorrect classifications of positive examples (**false negative**)
- ***FP*** – the number of incorrect classifications of negative examples (**false positive**)
- ***TN*** – the number of correct classifications of negative examples (**true negative**)

Precision and recall measures (cont…)

	Classified Positive	Classified Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

$$p = \frac{TP}{TP + FP} \qquad r = \frac{TP}{TP + FN}$$

- **Precision *p*** is the number of correctly classified positive examples divided by the total number of examples that are classified as positive.
- **Recall *r*** is the number of correctly classified positive examples divided by the total number of actual positive examples in the test set.

An example

	Classified Positive	Classified Negative
Actual Positive	1	99
Actual Negative	0	1000

- **This confusion matrix yields**
  - precision p = 100% and
  - recall r = 1%

  because we only classified one positive example correctly and no negative examples wrongly.
- **Note:** precision and recall measure classification only on the positive class.

F1-value (also called F1-score)

- It is hard to compare two classifiers using two measures. F_1 score combines precision and recall into one measure

$$F_1 = \frac{2pr}{p+r}$$

F1-score is the harmonic mean of precision and recall

$$F_1 = \frac{2}{\frac{1}{p} + \frac{1}{r}}$$

- The harmonic mean of two numbers tends to be closer to the smaller of the two.
- For F_1-value to be large, both *p* and *r* much be large.

Another evaluation method: Scoring and ranking

- **Scoring** is related to classification.
- We are interested in a single class (**positive class**), e.g., buyers class in a marketing database.
- Instead of assigning each test instance a definite class, scoring assigns a probability estimate (PE) to indicate the likelihood that the example belongs to the positive class.

Ranking and lift analysis

- After each example is given a PE score, we can rank all examples according to their PEs.
- We then divide the data into n (say 10) bins. A lift curve can be drawn according how many positive examples are in each bin. This is called **lift analysis**.
- Classification systems can be used for scoring. Need to produce a probability estimate.
  - E.g., in decision trees, we can use the confidence value at each leaf node as the score.

An example

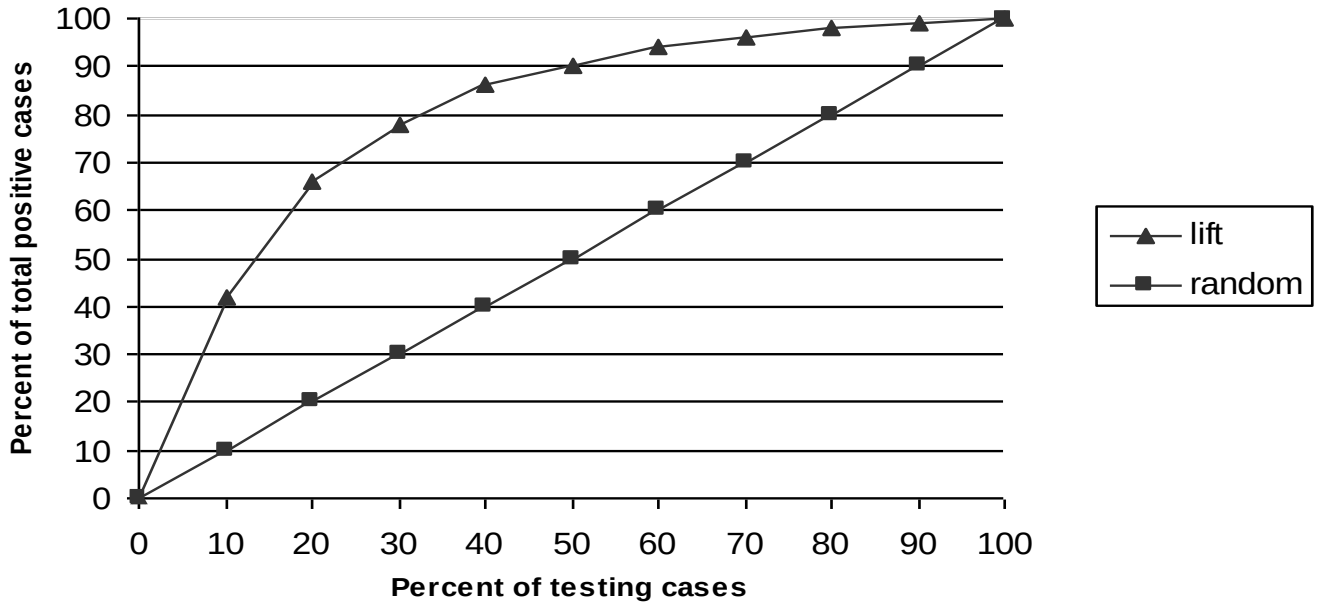
- We want to send promotion materials to potential customers to sell a watch.
- Each package cost $0.50 to send (material and postage).
- If a watch is sold, we make $5 profit.
- Suppose we have a large amount of past data for building a predictive/classification model. We also have a large list of potential customers.
- How many packages should we send and whom should we send to?

An example

- Assume that the test set has 10000 instances.
  Out of them, 500 are positive cases.
- After the classifier is built, we score each test instance. We then rank the test set and divide the ranked test set into 10 bins.
  - Each bin has 1000 test instances.
  - Bin 1 has 210 actual positive instances
  - Bin 2 has 120 actual positive instances
  - Bin 3 has 60 actual positive instances
  - …
  - Bin 10 has 5 actual positive instances

Lift curve

Bin |	1	2	3	4	5	6	7	8	9	10
	210	120	60	40	22	18	12	7	6	5
	42%	24%	12%	8%	4.40%	3.60%	2.40%	1.40%	1.20%	1%
	42%	66%	78%	86%	90.40%	94%	96.40%	97.80%	99%	100%



The ROC-curve (Receiver Operating Characteristic Curve)

A receiver operating characteristic (ROC) curve:

- commonly used to evaluate classification results on the positive class in two-class classification problems.

~ **a plot of the true positive rate (TPR) against the false positive rate (FPR)**

- $TPR = \frac{TP}{TP+FN}$ (the fraction of actual positive cases that are correctly classified)

  ~ TPR is essentially the recall of the positive class; it is also called **sensitivity**

- $FPR = \frac{FP}{TN+FP}$ (the fraction of actual negative cases classified to positive class)

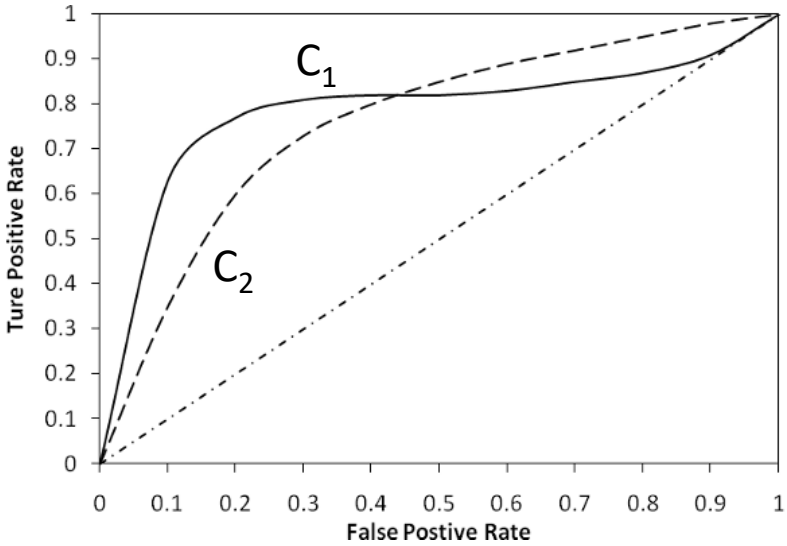
- $TNR = \frac{TN}{TN+FP}$ (true negative rate ~ the recall of the negative class)

  ~ another useful measure, sometimes called **specificity** ,

➔ ***FPR = 1 - specificity***

An example:

ROC curves for two classifiers (C 1 and C 2) on the same data



- each ROC-curve starts from (0, 0) and ends at (1, 1).
  - (0, 0) represents the situation where every test case is classified as negative,
  - (1, 1) represents the situation where every test case is classified as positive.
- the main diagonal line represents random guessing, i.e., predicting each case to be positive with a fixed probability.
  - in this case, TPR has the same value like FPT for each of the values, i.e., TPR = FPT.

An example:

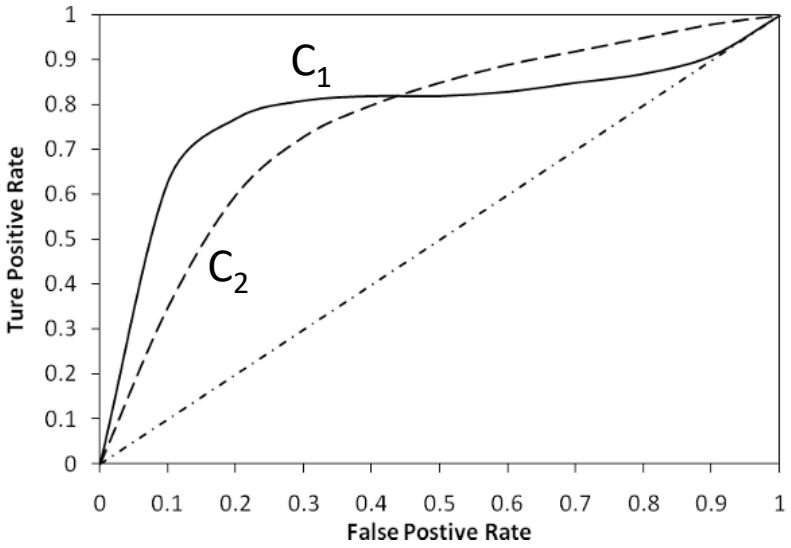
ROC curves for two classifiers (C 1 and C 2) on the same data

Question:

Which classifier is better – C_1 or C_2?

Answer:

- when FPR is less than 0.43, C_1 is better,
- when FPR is greater than 0.43, C_2 is better



X overall, the **area under the ROC curve (AUC)** can be used:

- if the AUC value for a classifier C_i is greater than that of another classifier C_j , it is said that ***C_i is better than C_j***.
- if a classifier is perfect, its AUC value is 1.
- if a classifier makes all random guesses, its AUC value is 0.5.

How to draw a ROC curve?

(given the classification as a ranking of test cases)

1. Obtain the ranking through sorting the test cases in a decreasing order of the classifier's output values (e.g., posterior probabilities).
2. Partition the rank list into two subsets at every test case and regard every test case in the upper part (with higher classifier output value) as a positive case and every test case in the lower part as a negative case.
3. For each such partition, compute a pair of TPR and FPR values.
   a. when the upper part is empty, we obtain the point (0, 0) on ROC
   b. when the lower part is empty, we obtain the point (1, 1).
4. Finally, connect the adjacent points.

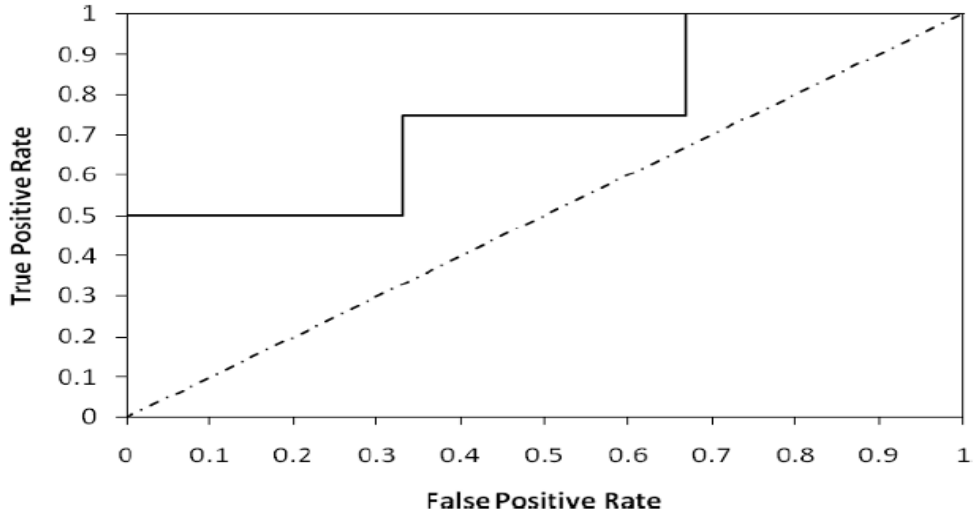
Example:
computations for drawing the ROC curve

- We have 10 test cases.
- A classifier has been built, and it has ranked the 10 test cases as shown in the second row of the Table below
  - the numbers in row 1 are the rank positions, with 1 being the highest rank and 10 the lowest.
  - The second row shows the actual class of each test case.
    - "+" means that the test case is from the positive class,
    - "–" means that it is from the negative class.

Rank		1	2	3	4	5	6	7	8	9	10
Actual class		+	+	–	–	+	–	–	+	–	–
TP	0	1	2	2	2	3	3	3	4	4	4
FP	0	0	0	1	2	2	3	4	4	5	6
TN	6	6	6	5	4	4	3	2	2	1	0
FN	4	3	2	2	2	1	1	1	0	0	0
TPR	0	0.25	0.5	0.5	0.5	0.75	0.75	0.75	1	1	1
FPR	0	0	0	0.17	0.33	0.33	0.50	0.67	0.67	0.83	1

Example:
computations for drawing the ROC curve

Rank		1	2	3	4	5	6	7	8	9	10
Actual class		+	+	–	–	+	–	–	+	–	–
TP	0	1	2	2	2	3	3	3	4	4	4
FP	0	0	0	1	2	2	3	4	4	5	6
TN	6	6	6	5	4	4	3	2	2	1	0
FN	4	3	2	2	2	1	1	1	0	0	0
TPR	0	0.25	0.5	0.5	0.5	0.75	0.75	0.75	1	1	1
FPR	0	0	0	0.17	0.33	0.33	0.50	0.67	0.67	0.83	1



Data Mining

ADVANCED DATA PRE-PROCESSING

BY: IVETA MRÁZOVÁ

Data pre-processing

- Of key importance for the success of the application
- Difficult and time-consuming
- Collaboration with a domain expert is of an advantage

The goal of pre-processing:

- Select (or create) from the available data those ones relevant for the data mining task at hand
- Represent this data in a form suitable for processing by the chosen algorithm
- **Final state** ~ (one) data table containing the attribute values of the objects

Structured data

Temporal data

- e.g., time series of stock prices
- A typical task ~ predict future values
- Example: a time series after transformation

INPUTS	OUTPUTS
$y(t_0), y(t_1), y(t_2), y(t_3)$	$y(t_4)$
$y(t_1), y(t_2), y(t_3), y(t_4)$	$y(t_5)$
$y(t_2), y(t_3), y(t_4), y(t_5)$	$y(t_6)$
.....	

Structured data (2)

Spatial data

- e.g., geographic information systems
- an implicit neighborhood relationship
  - Attribute values of „adjacent“ objects will not differ too much one from the other
- Useful especially for interpretation

Structural data

- e.g., chemical compounds
- Notation, e.g., by means of the so-called SMILES strings or using files of facts in Prolog
- e.g., classification of chemicals into classes based on their structure

Data with too many objects

Problematic processing in batch regimen!

- use only a sample set of objects selected from the data
- use such a way to store the data that would enable to access all the objects without the necessity to save them to the operating memory
- build more models based on the subsets of objects and combine the models afterwards

➔ representativeness of the selected data

- the selected objects should capture the nature of the data as best as possible (concerning, e.g., the correspondence of the attribute value distribution both on the selected sample set and the entire data)

Data with too many objects (2)

- ✖ unbalanced classes in the original data
  ($\Rightarrow$ tendency to prefer the majority class)
  - ➢ different weights for different types of misclassification
  - ➢ select examples from different classes with a different probability

➔ Cross -Validation

- ➢ From the data, we select just its certain part for training and another (different) part for testing

➔ more efficient data storage and processing (parallelism)

Data with too many attributes

- reduce the number of attributes with an expert
- automatic reduction of the number of the attributes
  - through **transformation** ~ from the existing attributes create less new attributes (e.g., PCA analysis, …)
    - new attributes are formed as a linear combination of original attributes
    - ✖ the necessity to provide the values of all original attributes
    - ✖ new attributes might lack a clear interpretation
  - through **selection** ~ from the existing attributes choose only the most important ones (Feature Selection)

Selection

- **we are looking for those attributes, that best contribute to a proper classification of objects**
  - **Filter methods** ~ to each of the attributes, compute a characteristics that reflects its contribution to classification
  - **Wrapper methods** ~ use a machine learning algorithm to build a model using a subset of the attributes and evaluate the model. Then, use the best of the built models (with the selected attributes).
    - **„bottom-up“** ~ begin from the models built for individual attributes and add other attributes subsequently
    - **„top-down“** ~ start with the model built for the complete set of attributes and remove the unnecessary ones subsequently

Automatic feature selection by a filter method

- **criteria based on contingency tables**

	$C(v_1)$	$C(v_2)$	…	$C(v_S)$	Σ
$A(v_1)$	a_{11}	a_{12}	…	a_{1S}	r_1
$A(v_2)$	a_{21}	a_{22}	…	a_{2S}	r_2
⋮	⋮	⋮	…	⋮	⋮
$A(v_R)$	a_{R1}	a_{R2}	…	a_{RS}	r_R
Σ	s_1	s_2	…	s_S	n

A input attribute

C target attribute

a_{kl} frequency of combination $(A(v_k) \wedge C(v_l))$

$$r_k = \sum_{l=1}^{S} a_{kl} \qquad s_l = \sum_{k=1}^{R} a_{kl} \qquad n = \sum_{k=1}^{R}\sum_{l=1}^{S} a_{kl}$$

Automatic feature selection by a filter method (2)

- **Probability estimation:**

$$P(A(v_k) \wedge C(v_l)) = \frac{a_{kl}}{n}$$

$$P(A(v_k)) = \frac{r_k}{n} \qquad P(C(v_l)) = \frac{s_l}{n}$$

- **criteria for attribute selection**
  - χ^2 (the higher the value, the better)

$$\chi^2 = \sum_{k=1}^{R}\sum_{l=1}^{S}\frac{(a_{kl} - e_{kl})^2}{e_{kl}} = n\sum_{k=1}^{R}\sum_{l=1}^{S}\frac{\left(a_{kl} - \frac{r_k s_l}{n}\right)^2}{\frac{r_k s_l}{n}}$$

Automatic feature selection by a filter method (3)

- **criteria to select the attributes**
  - **entropy *H (A)*** (the smaller the value, the better)

$$H(A) = \sum_{k=1}^{R} \frac{r_k}{n} \, H\big(A(v_k)\big)$$

where $$H\big(A(v_k)\big) = -\sum_{l=1}^{S} \frac{a_{kl}}{r_k} \log_2 \frac{a_{kl}}{r_k}$$

Automatic feature selection by a filter method (4)

criteria to select the attributes (continued)

- **information measure of dependence ID(A,C)** (the bigger the value, the better)

$$ID(A,C) = \frac{MI(A,C)}{H(C)} = \frac{MI(A,C)}{-\sum_{l=1}^{S} \frac{s_l}{n} \log \frac{s_l}{n}}$$

- where **mutual information MI(A,C)**:

$$MI(A,C) = \sum_{k=1}^{R} \sum_{l=1}^{S} P(A(v_k) \wedge C(v_l)) \log_2 \frac{P(A(v_k) \wedge C(v_l))}{P(A(v_k))\,P(C(v_l))}$$

$$= \sum_{k=1}^{R} \sum_{l=1}^{S} \frac{a_{kl}}{n} \log_2 \frac{\frac{a_{kl}}{n}}{\frac{r_k}{n}\frac{s_l}{n}} = \frac{1}{n} \sum_{k=1}^{R} \sum_{l=1}^{S} a_{kl} \log_2 \frac{n a_{kl}}{r_k s_l}$$

Automatic feature selection by a filter method (5)

- the attributes can be ordered according to the considered criteria values and then, a certain number of the best attributes is selected
  - approach „bottom-up“ (~ add attributes)
  - approach „top-down“ (~ remove attributes)

✖ each attribute is evaluated separately

✖ mutual influence of several attributes on the accuracy of the classification is difficult to be captured

- **compute the value of the criterion for a set of attributes** (~ for all combinations of these attributes)
  - **selection „bottom-up“** (~ adding of attributes – ***direct selection***)
  - **proceed „top-down“** (~ elimination of attributes – ***backward selection***)

Numeric attributes

Discretization of numeric attributes (~ division to intervals)

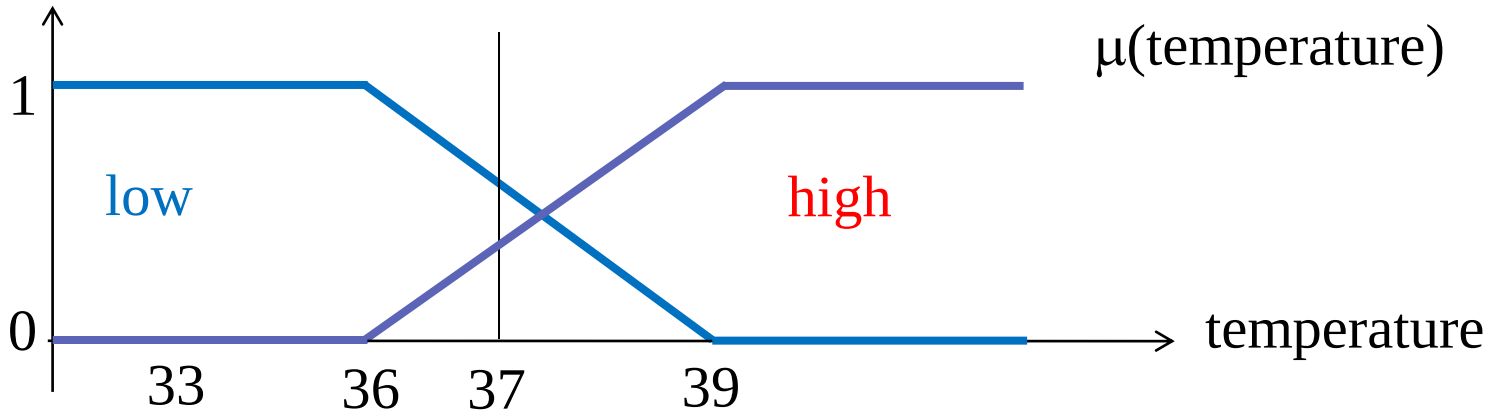
- **discretization to a preset number of equidistant intervals** (~ the range of the attribute values will be divided into intervals of the same length)
- **discretization by using the information on class membership of the objects**
- the methods differ in:
  - the strategy adopted to create the intervals (a top-down division of the intervals, or a bottom-up joining of the intervals, resp.)
  - the number of the obtained intervals
  - the type of the intervals
  - the criterion used to express the quality of the intervals (minimum classification error, entropy, χ^2-test)

Fuzzy discretization

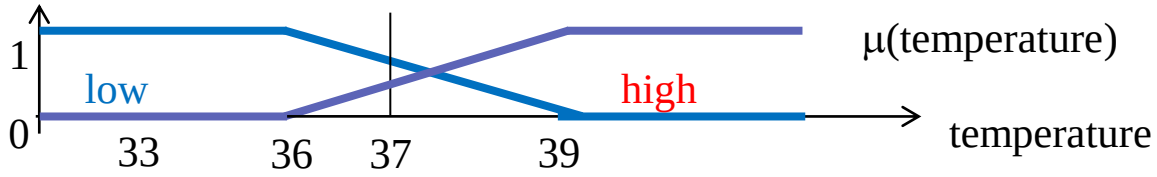
- the boundaries of fuzzy-intervals are ‘blurry’

  $\Rightarrow$ provide a relevant procedure **‘Interval’** to find appropriate input value intervals by concatenating sequences of the input values with the same (fuzzy-)class

- **fuzzy-intervals**



Fuzzy discretization (2)



- to one numeric value, we can obtain two discretized values, such that the sum of the corresponding characteristic functions will be equal to 1
  - arise 'fuzzy'-objects with weights < 1
  - the weights of 'fuzzy'-objects are given by the product of the values of characteristic functions of all the attributes:

$$w(obj) = \Pi_i \, \mu(x_i)$$

  - possible loss of information
  - **contradiction** ~ objects with the same values of the input (discretized) attributes belong to different classes

Fuzzy discretization: Procedure 'Interval'

3.1. if the sequence of values has the same class label, then build an interval of these values (with the characteristic function equal to 1 on the entire interval)

3.2. for each interval $\boldsymbol{Int_i}$

if interval $\boldsymbol{Int_i}$ belongs to class **'?'**

then if its adjacent intervals Int_{i-1} and Int_{i+1} belong to the same class

3.2.1. build an interval by joining $\boldsymbol{Int_{i-1} \cup Int_i \cup Int_{i+1}}$ (with the characteristic function equal to 1 on the entire interval)

3.2.2. else join $\boldsymbol{Int_{i-1} \cup Int_i}$ and join $\boldsymbol{Int_i \cup Int_{i+1}}$ to form two fuzzy-intervals

/ continue on the next slide

Fuzzy discretization: Procedure 'Interval' (continued)

3.2.2. else form one interval by joining $\boldsymbol{Int_{i-1} \cup Int_i}$ such that the characteristic function is equal to 1 on the interval $\boldsymbol{[Llim_{i-1}, Ulim_{i-1}]}$ and equal to:

$$\frac{Llim_{i+1} - x}{Llim_{i+1} - Ulim_{i-1}} \quad \text{for} \quad x \in [Ulim_{i-1}, Llim_{i+1}]$$

and another interval by joining $\boldsymbol{Int_i \cup Int_{i+1}}$ such that the characteristic function is equal to 1 on the interval $\boldsymbol{[Llim_{i+1}, Ulim_{i+1}]}$ and equal to:

$$\frac{x - Ulim_{i-1}}{Llim_{i+1} - Ulim_{i-1}} \quad \text{for} \quad x \in [Ulim_{i-1}, Llim_{i+1}]$$

3.3. provide a continuous coverage of the domain of the interval values similarly to Step 3.2.2. (the gaps between the intervals are understood as intervals of class **'?'**)

Categorical attributes

Algorithm to group the values (KEX):

if the categorical attribute has too many values, it might be **useful to group them** together (e.g., based on the characteristics computed on the training data)

⇒ **Objective:**

form groups ***A*(*Grp*)** such that ***P*(*Class* | *A*(*Grp*))** significantly differs from ***P*(*Class*)**

+ form so many groups that we have classes plus one more(´**?**´)

Categorical attributes (2)

Algorithm to group the values (KEX):

The main cycle:

1. build an ordered list of the considered attribute values
2. for each value
   - 2.1. compute the frequencies of occurrence of the objects with this value for the respective classes
   - 2.2. assign a class code to each of the values by means of the procedure **Assign**
3. form the intervals of the values by means of procedure **Group**

Categorical attributes (3)

Algorithm to group the values (KEX):

Procedure Assign:

If all the objects belong to the same class for the considered value

1. then assign the code of this class to the value;
2. else – if the object class distribution significantly differs for the considered value (based on the χ^2-test) from the apriori class distribution
    a) Then assign the code of the most frequent class to the value
    b) Else assign the code **'?'** to the value

Procedure Group:

form a group of the values that were assigned the same class code

Missing values

- ignore objects with any of the values missing
- replace the missing value by a new value **‘I don´t know’**
- replace the missing value by an existing attribute value
  - the most frequent value
  - proportional to the ratio of all values
  - any value
- completion of missing values by means of (a used) model